

SESIÓN 2: DATA WRANGLING Y PIVOTS

Data Wrangling y Pivots

Fausto Jácome Pérez
FLACSO Ecuador

Tengo los datos en memoria. ¿Cómo los preparo para responder una pregunta sobre desnutrición infantil?

¿Dónde estamos?

1

SESIÓN ANTERIOR

Cargamos `f1_personas`. Vimos el schema, los tipos inferidos, los missings. Terminamos con una pregunta pendiente: ¿por qué `area` es `Float64`?

2

ESTA SESIÓN

Corregir tipos. Recodificar. Filtrar y seleccionar. Calcular edad en días. Joins.

3

PRÓXIMA SESIÓN

SQL: las mismas operaciones en otro lenguaje, y por qué conviene conocer los dos.

El objetivo de hoy: dejar `f1_personas` lista para cruzar con `f1_hogar` y avanzar hacia el cálculo de prevalencia de desnutrición crónica.

Paso 0: tipos de datos

Antes de cualquier operación, los tipos tienen que ser correctos

El problema de los tipos mal inferidos

Cuando Polars lee un archivo desde R, hereda los tipos asignados por `pyreadr`. Algunos son incorrectos:

Variable	Tipo inferido	Tipo correcto	Por qué importa
<code>area</code>	<code>Float64</code>	<code>Categorical</code>	Urbano=1, Rural=2 no son magnitudes
<code>region</code>	<code>Float64</code>	<code>Categorical</code>	Sierra=1, Costa=2, Amazonia=3
<code>prov</code>	<code>Float64</code>	<code>String</code>	Código de provincia, no número
<code>f1_s1_2</code>	<code>Int32</code>	<code>Categorical</code>	Sexo: 1=Hombre, 2=Mujer
<code>grupo_edad_nin</code>	<code>Float64</code>	<code>Enum</code>	Grupos ordenados de edad

Un `Float64` donde debería haber una categoría no genera error: simplemente devuelve resultados sin sentido. `df["area"].mean()` da 1.38. Nadie lo advierte.

Recodificación con `pl.when().then().otherwise()`

Mapeamos los códigos a etiquetas y declaramos el tipo correcto en un solo paso:

```
df = df.with_columns([\n\n    # area: nominal - sin orden entre categorías\n    pl.when(pl.col("area") == 1).then(pl.lit("Urbano"))\n      .when(pl.col("area") == 2).then(pl.lit("Rural"))\n      .otherwise(pl.lit(None))\n      .cast(pl.Categorical)\n      .alias("area"),\n\n])
```

pl.Categorical VS pl.Enum

pl.Categorical

Categorías descubiertas al leer los datos. Sin orden definido.

Usar para variables **nominales**: `area`, `region`, `sexo`, `etnia`.

```
.cast(pl.Categorical)
```

pl.Enum

Categorías declaradas explícitamente y en orden. Lista cerrada.

Usar para variables **ordinales**: grupos de edad, quintiles, nivel educativo.

```
orden = ["0-5 meses", "6-11 meses",  
        "12-23 meses", "24-35 meses",  
        "36-47 meses", "48-59 meses"]  
  
.cast(pl.Enum(orden))
```

`pl.Enum` lanza un error si encuentra un valor que no está en la lista declarada. Es una ventaja: actúa como validación automática. Sin `pl.Enum`, al ordenar por grupo de edad Polars usaría orden alfabético: `"24-35 meses"` antes que `"0-5 meses"`.

Selectores - referencia rápida

Los selectores (`cs`) describen *grupos de columnas* por tipo, nombre o patrón:

```
import polars.selectors as cs
```

Por tipo de dato

```
cs.numeric()      # Int*, Float*, UInt*
cs.string()       # Utf8 / String
cs.boolean()      # Bool
cs.temporal()     # Date, Datetime, Duration, Time
cs.categorical()  # Categorical
```

Por nombre o patrón

```
cs.starts_with("f1_s5_")
cs.ends_with("_id", "_cod")
cs.contains("monto", "valor")
cs.by_name("edad", "sexo")
```

Condiciones compuestas

```
# Numéricas excepto identificadores
cs.numeric() & ~cs.ends_with("_id")

# Variables de monto o de fecha
cs.contains("monto") | cs.temporal()

# Todo excepto strings
~cs.string()
```

Selectores en `.with_columns()` - caso ENDI

El caso más útil en encuestas: aplicar la misma transformación a un grupo de columnas sin repetir una expresión por cada una.

Reemplazar códigos de no respuesta en todas las columnas de una sección

```
# En la ENDI, 88 = "No sabe", 99 = "No responde", 9999 = omisión
df = df.with_columns(
    cs.starts_with("f1_s1_").replace({88: None, 99: None, 9999: None})
)
```

Cast de todos los identificadores en una sola línea

```
df = df.with_columns(
    cs.by_name("id_upm", "id_viv", "id_hogar", "id_per", "id_mef")
    .cast(pl.String)
)
```

Regla práctica: selector cuando el criterio es estructural (tipo, prefijo, sufijo). Lista explícita cuando el criterio es semántico: no todas las columnas que empiezan con `f1_s1_` son categóricas.

Ejercicio 1

Recodificar variables y corregir tipos

Ejercicio 1 - Recodificación

Objetivo: aplicar la recodificación a variables de `f1_personas` usando el diccionario de variables de la ENDI.

Abran el notebook `02_ejercicios_wrangling.ipynb` y ejecuten hasta la celda de recodificación.

Lo que deben completar:

1. Recodificar `area`, `region`, `f1_s1_2` y `etnia` con `pl.when().then()` y `.cast(pl.Categorical)`
2. Recodificar `grupo_edad_nin` con `.cast(pl.Enum(orden_grupos))`
3. Verificar con `df.select(...).schema` que los tipos quedaron correctos

Pregunta de cierre: ¿por qué `prov` va a `pl.String` y no a `pl.Categorical`? ¿En qué caso sería útil usarla como `Categorical`?

Filtros y selección

`.filter()`, `.select()`, `.drop()` y selectores

`.filter()` - filas que cumplen una condición

```
# Menores de 5 años (< 1826 días) - población objetivo de desnutrición crónica
menores5 = df.filter(pl.col("edaddias") < 1826)

# Menores de 2 años (< 730 días) - grupo objetivo del Decreto Presidencial 1211
menores2 = df.filter(pl.col("edaddias") < 730)

# Condiciones múltiples con & (and) y | (or)
urbanos_menores5 = df.filter(
    (pl.col("area") == pl.lit("Urbano")) &
    (pl.col("edaddias") < 1826)
)

# Filtrar por grupo_edad_nin no nulo - forma más directa para menores con medición
df_nin = df.filter(pl.col("grupo_edad_nin").is_not_null())
```

Con `Categorical`, comparar contra strings requiere `pl.lit()`. Sin él, Polars puede lanzar un error de tipos o comparar de forma inesperada.

`.select()` y `.drop()` - columnas que necesito

`.select()` elige columnas. `.drop()` descarta. Ambas admiten selectores:

```
# Selección manual
df_trabajo = df.select(["id_per", "id_hogar", "id_upm"])

# Con selectores: IDs + (numéricas + patrón de inicio)
df_trabajo = df.select(
    cs.by_name("id_per", "id_hogar", "id_upm", "fexp", "estrato") |
    (cs.numeric() &
     cs.starts_with("f1_s1_7"))
)

# Si quiero eliminar
df_trabajo = df_trabajo.drop(["id_per", "id_hogar", "id_upm"])
```

Piedra Rosetta - Cast y Filtros

Python · Polars

```
# Recodificar
df = df.with_columns(
    pl.when(pl.col("area") == 1)
        .then(pl.lit("Urbano"))
        .when(pl.col("area") == 2)
        .then(pl.lit("Rural"))
        .otherwise(pl.lit(None))
        .cast(pl.Categorical)
        .alias("area")
)

# Filtrar
df_nin = df.filter(
    pl.col("grupo_edad_nin")
        .is_not_null()
)
```

R · dplyr / sjlabelled

```
library(dplyr)
library(sjlabelled)

# En R la ENDI ya trae etiquetas
df <- df %>%
  mutate(area = as_label(area))

# O manual con case_when
df <- df %>%
  mutate(
    area = case_when(
      area == 1 ~ "Urbano",
      area == 2 ~ "Rural",
      TRUE ~ NA_character_
    )
  )

# Filtrar
df_nin <- df %>%
  filter(!is.na(grupo_edad_nin))
```

SQL · PostgreSQL

```
-- Recodificar al consultar
SELECT
  CASE area
    WHEN 1 THEN 'Urbano'
    WHEN 2 THEN 'Rural'
    ELSE NULL
  END AS area,
  region,
  edaddias
FROM personas
-- Filtrar
WHERE grupo_edad_nin
  IS NOT NULL;
```

Ejercicio 2

Seleccionar y filtrar

Ejercicio 2 - Select y filter

Objetivo: reducir el dataframe a las columnas y filas necesarias para el indicador.

Continúen en el notebook:

1. Apliquen `.select()` para quedarse con las columnas del indicador: identificadores, variables de desagregación, `fexp`, `edaddias`, `grupo_edad_nin`, `dcronica`, `dcronica2_5`
2. Filtren con `grupo_edad_nin.is_not_null()` para quedarse con los menores de 5 años que tienen medición
3. Verifiquen: ¿cuántas filas quedan? ¿Tiene sentido ese número?

Pregunta de cierre: si en lugar de `.select()` usaran `.drop()` para llegar al mismo resultado, ¿cuál de los dos sería más cómodo para esta tabla de 124 columnas?

Fechas y edad en días

Construir variables a partir de columnas crudas

Calcular edad en días desde componentes

La ENDI guarda día, mes y año en columnas separadas. `pl.date()` las combina:

```
df_nin = df_nin.with_columns([
    pl.date(
        pl.col("f1_s5_2_3").cast(pl.Int32), # año nacimiento
        pl.col("f1_s5_2_2").cast(pl.Int32), # mes nacimiento
        pl.col("f1_s5_2_1").cast(pl.Int32), # día nacimiento
    ).alias("fecha_nac"),

    pl.date(
        pl.col("f1_s5_3_3").cast(pl.Int32), # año medición
        pl.col("f1_s5_3_2").cast(pl.Int32), # mes medición
        pl.col("f1_s5_3_1").cast(pl.Int32), # día medición
    ).alias("fecha_med"),
])

# Edad en días como entero
df_nin = df_nin.with_columns(
    (pl.col("fecha_med") - pl.col("fecha_nac"))
    .dt.total_days()
    .alias("edad_dias_calc")
)
```

Comparar con la variable precalculada

La tabla ya tiene `edaddias` calculada por el INEC. Podemos verificar que nuestra lógica coincide:

```
print(
    df_nin.select([
        "edaddias",
        "edad_dias_calc",
        (pl.col("edaddias") - pl.col("edad_dias_calc"))
            .abs()
            .alias("diferencia")
    ]).describe()
)

# ¿Hay casos donde difieren más de 1 día?
distintos = df_nin.filter(
    (pl.col("edaddias") - pl.col("edad_dias_calc")).abs() > 1
)
print(f"Casos con diferencia > 1 día: {distintos.shape[0]}")
```

429 casos difieren en más de 1 día. La causa más probable es que el INEC aplica una corrección adicional para fechas incompletas o inconsistentes en campo. Para el curso usamos la variable `edaddias` precalculada.

Ejercicio 3

Calcular edad en días y comparar

Ejercicio 3 - Fechas

Objetivo: construir `fecha_nac`, `fecha_med` y `edad_dias_calc` y comparar con `edaddias`.

Continúen en el notebook:

1. Construyan `fecha_nac` y `fecha_med` con `pl.date()`
2. Calculen `edad_dias_calc` con `.dt.total_days()`
3. Comparen con `edaddias` del INEC: ¿cuántos casos difieren en más de 1 día?
4. Explore uno de esos casos: ¿qué tiene diferente?

Pregunta de cierre: ¿por qué el INEC calcula la edad en días y no en años o meses? ¿Qué información se pierde si se usa edad en meses para definir el denominador del indicador de desnutrición crónica?

Joins

Combinar tablas relacionales

¿Por qué necesitamos joins?

La ENDI está organizada en seis tablas relacionales. Cada una tiene su unidad de análisis:

`f1_personas` → persona

`f1_hogar` → hogar

`f2_mef` → mujer en edad fértil

Comparten llaves: `id_hogar` , `id_per` , `id_mef` .

Para cruzar información necesitamos un **join**: combinar filas de dos tablas usando una columna como llave.

El tipo de join define qué filas se conservan cuando no hay coincidencia.

Caso concreto: queremos agregar el tipo de vivienda y la fuente de agua del hogar a la tabla de niños. Esa información está en `f1_hogar` , no en `f1_personas` .

Tipos de join

Tipo	¿Qué conserva?	Cuándo usarlo en la ENDI
left	Todas las filas de la tabla izquierda	Agregar columnas de hogar a personas sin perder personas
inner	Solo filas con coincidencia en ambas	Subconjunto que existe en las dos tablas
full	Todas las filas de ambas tablas	Diagnóstico: ver qué falta en cada lado
semi	Filas izquierda con coincidencia, sin agregar columnas	Filtrar personas que tienen hogar registrado
anti	Filas izquierda sin coincidencia	Encontrar personas sin hogar registrado

[Ver el cheat sheet](#)

Left join en Polars

```
# Cargar f1_hogar y seleccionar las columnas de interés
df_hogar = pl.from_pandas(pyreadr.read_r(ruta_hogar)[None])

df_hogar_min = df_hogar.select(["id_hogar", "f1_s3_2", "f1_s7_1"]).rename({
    "f1_s3_2": "tipo_vivienda",
    "f1_s7_1": "fuente_agua",
})

# Left join: agregar variables del hogar a los niños
df_nin_hogar = df_nin.join(df_hogar_min, on="id_hogar", how="left")
```

El número de filas no cambia con un `left join` : todos los niños se conservan. Si algún hogar no tiene registro en `f1_hogar` , las columnas nuevas quedan como `null` .

Verificar el join - inner vs anti

Una buena práctica después de cualquier join es verificar que el resultado tiene sentido:

```
n_left = df_nin.join(df_hogar_min, on="id_hogar", how="left").shape[0]
n_inner = df_nin.join(df_hogar_min, on="id_hogar", how="inner").shape[0]
n_anti = df_nin.join(df_hogar_min, on="id_hogar", how="anti").shape[0]

print(f"left join: {n_left:>7,} (conserva todos los niños)")
print(f"inner join: {n_inner:>7,} (solo niños con hogar coincidente)")
print(f"anti join: {n_anti:>7,} (niños sin hogar coincidente)")
print(f"inner + anti = {n_inner + n_anti:,} (debe coincidir con el total)")
```

Si `inner + anti ≠ left`, hay un problema: algún niño tiene más de un hogar coincidente y el join está multiplicando filas. Eso indica una llave duplicada en la tabla derecha.

Ejercicio 4

Join de personas con hogar

Ejercicio 4 - Left join

Objetivo: agregar `tipo_vivienda` y `fuente_agua` de `f1_hogar` a la tabla de niños.

Continúen en el notebook:

1. Carguen `f1_hogar` y seleccionen `id_hogar`, `f1_s3_2` y `f1_s7_1`
2. Renombren las columnas a `tipo_vivienda` y `fuente_agua`
3. Apliquen el `left join` sobre `id_hogar`
4. Verifiquen con `inner + anti` que el resultado es consistente
5. ¿Cuántos niños viven en hogares con agua de red pública? (código 1 en `fuentes_agua`)

Reshaping: Long vs Wide

La forma de la tabla determina las preguntas que puedes responder

El problema de la tabla MEF

La tabla `f2_mef` registra información de mujeres en edad fértil. Cada mujer puede tener varios hijos. El INEC guardó los datos en formato **wide**: cada hijo es un conjunto de columnas.

Estructura wide (lo que tiene la tabla):

id_mef	f2_s2_235_a_01	f2_s2_235_b_dia_01	f2_s2_235_a_02	f2_s2_235_b_dia_02	...
M001	1 (Hombre)	15	2 (Mujer)	3	...
M002	2 (Mujer)	22	null	null	...

El problema: para calcular la edad de vacunación de cada hijo necesito una fila por hijo, no una columna por hijo.

En formato wide no se puede filtrar *"todos los hijos menores de 1 año"* con un solo `.filter()`. Habría que escribir una condición por cada columna de hijo.

Wide a Long - concepto

Wide: cada observación es una fila. Las categorías de una variable se distribuyen en columnas.

```
id_mef | hijo_01_sexo | hijo_02_sexo
M001   | Hombre         | Mujer
M002   | Mujer          | null
```

Útil para: presentación, tablas cruzadas, atributos fijos y pocos.

Long: cada combinación (unidad, categoría) es una fila.

```
id_mef | ord_hijo | sexo
M001   | 01       | Hombre
M001   | 02       | Mujer
M002   | 01       | Mujer
```

Útil para: análisis, filtros, cálculos, gráficos. Es la forma **tidy**.

La mayoría de las operaciones de análisis esperan datos en formato **long**. El formato wide es para leer, el long es para calcular.

`.unpivot()` y `.pivot()`

```
# Wide a long
cols_hijos = [c for c in df_mef.columns if c.startswith("f2_s2_235_")]

df_long = df_mef.select(["id_mef"] + cols_hijos).unpivot(
    on=cols_hijos,
    index=["id_mef"],
    variable_name="campo",
    value_name="valor",
)

# Long a wide
df_wide = df_long.pivot(
    on="campo",
    index=["id_mef"],
    values="valor",
    aggregate_function="first"
)
```


Piedra Rosetta - Reshaping

Python · Polars

```
# Wide a Long
df_long = df_wide.unpivot(
    on=cols_variables,
    index=["id_mef"],
    variable_name="campo",
    value_name="valor"
)

# Long a wide
df_wide2 = df_long.pivot(
    on="campo",
    index=["id_mef"],
    values="valor",
    aggregate_function="first"
)
```

R · tidyr

```
library(tidyr)

# Wide a Long
df_long <- df_wide %>%
  pivot_longer(
    cols = starts_with("f2_s2_235_"),
    names_to = "campo",
    values_to = "valor"
)

# Long a wide
df_wide2 <- df_long %>%
  pivot_wider(
    id_cols = id_mef,
    names_from = campo,
    values_from = valor
  )
```

¿Qué aprendimos hoy?

Conceptos

- Los tipos mal inferidos son errores silenciosos
- `pl.Categorical` para variables nominales, `pl.Enum` para ordinales
- Selectores (`cs`) para operar sobre grupos de columnas
- Filtros con condiciones múltiples
- Long vs wide: el formato determina las preguntas que puedes responder

En Polars

- `pl.when().then().otherwise().cast().alias()`
- `cs.by_name()` , `cs.starts_with()` , `cs.categorical()`
- `.filter()` con `pl.col()` y `pl.lit()`
- `.select()` y `.drop()` con selectores
- `pl.date()` , `.dt.total_days()`
- `.join(df2, on=..., how=...)`

Tarea: a partir de `f1_personas` , aplicar los casts necesarios, filtrar menores de 5 años, y calcular la prevalencia de `dcronica` por área como promedio simple con `.group_by().agg(pl.mean())` . El resultado debe ser una tabla de dos filas.